Machine Learning: Unsupervised Learning

Part – 1

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com

Content

- What is K-Means Algorithm?
- How does the K-Means Algorithm Work?
- Challenges With the K-Means Clustering Algorithm
- K-means++
- Choosing right K
- Python Implementation

What is K-Means Algorithm?

What is Clustering?

- Cluster analysis is a technique in data mining and machine learning that groups similar objects into clusters. K-means clustering, a popular method, aims to divide a set of objects into K clusters, minimizing the sum of squared distances between the objects and their respective cluster centers.

- Hierarchical clustering and k-means clustering are two popular techniques in the field of unsupervised learning used for clustering data points into distinct groups. While k-means clustering divides data into a predefined number of clusters, hierarchical clustering creates a hierarchical tree-like structure to represent the relationships between the clusters.

What is K-Means Algorithm?

- K-means clustering is a popular unsupervised machine learning algorithm used for partitioning a dataset into a pre-defined number of clusters. The goal is to group similar data points together and discover underlying patterns or structures within the data.

- Recall the first property of clusters – it states that the points within a cluster should be similar to each other. So, our aim here is to minimize the distance between the points within a cluster.

- K-means is a centroid-based algorithm or a distance-based algorithm, where we calculate the distances to assign a point to a cluster. In K-Means, each cluster is associated with a centroid.

What is K-Means Algorithm?

- The main objective of the K-Means algorithm is to minimize the sum of distances between the points and their respective cluster centroid.

- Optimization plays a crucial role in the k-means clustering algorithm. The goal of the optimization process is to find the best set of centroids that minimizes the sum of squared distances between each data point and its closest centroid.

Content

- ~~What is K-Means Algorithm?~~
- How does the K-Means Algorithm Work?
- Challenges With the K-Means Clustering Algorithm
- K-means++
- Choosing right K
- Python Implementation

How does the K-Means Algorithm Work?

1. Initialization: Start by randomly selecting K points from the dataset. These points will act as the initial cluster centroids.

2. Assignment: For each data point in the dataset, calculate the distance between that point and each of the K centroids. Assign the data point to the cluster whose centroid is closest to it. This step effectively forms K clusters.

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where:

- C_k = set of points assigned to cluster k
- μ_k = centroid (mean) of cluster k

How does the K-Means Algorithm Work?

3. Update centroids: Once all data points have been assigned to clusters, recalculate the centroids of the clusters by taking the mean of all data points assigned to each cluster.

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

where:

- C_k = set of points assigned to cluster k
- μ_k = centroid (mean) of cluster k

How does the K-Means Algorithm Work?

4. Repeat: Repeat steps 2 and 3 until convergence. Convergence occurs when the centroids no longer change significantly or when a specified number of iterations is reached.

5. Final Result: Once convergence is achieved, the algorithm outputs the final cluster centroids and the assignment of each data point to a cluster.

How does the K-Means Algorithm Work?

Objective of k means Clustering

- The main objective of k-means clustering is to partition your data into a specific number (k) of groups, where data points within each group are similar and dissimilar to points in other groups.

- It achieves this by minimizing the distance between data points and their assigned cluster's center, called the centroid.

Content

- ~~What is K-Means Algorithm?~~
- ~~How does the K-Means Algorithm Work?~~
- Challenges With the K-Means Clustering Algorithm
- K-means++
- Choosing right K
- Python Implementation

Challenges With the K-Means

Challenges

- The k value in k-means clustering is a crucial parameter that determines the number of clusters to be formed in the dataset.

- Finding the optimal k value in the k-means clustering can be very challenging, especially for noisy data. The appropriate value of k depends on the data structure and the problem being solved.

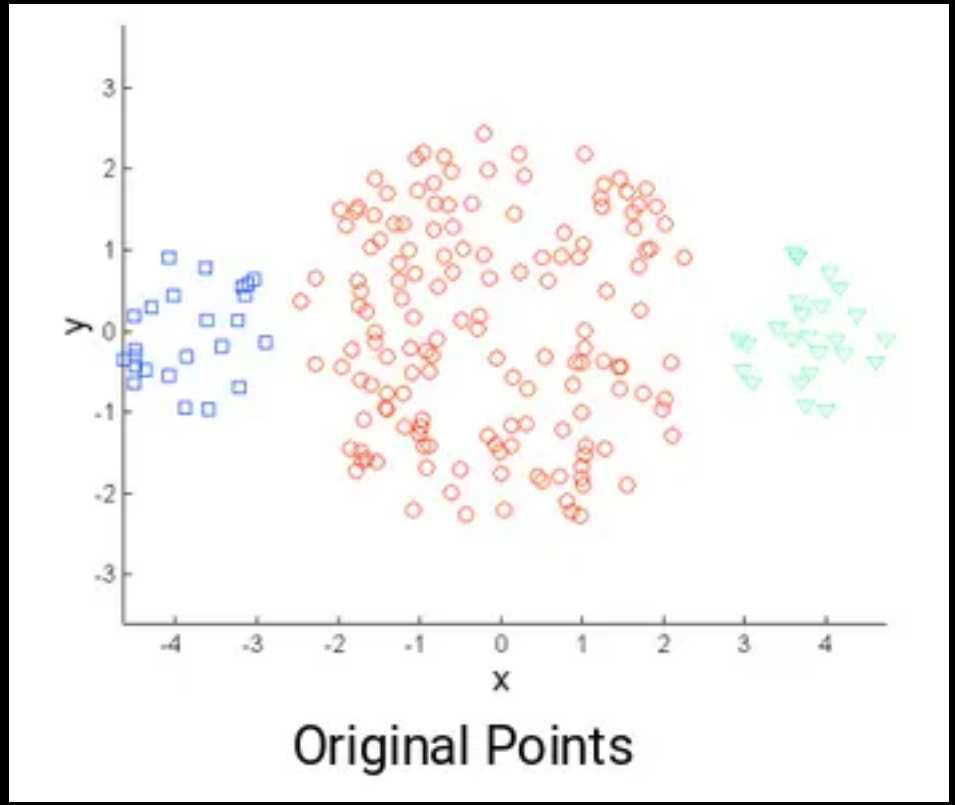
- It is important to choose the right value of k, as a small value can result in under-clustered data, and a large value can cause over-clustering.

Challenges With the K-Means

Challenges

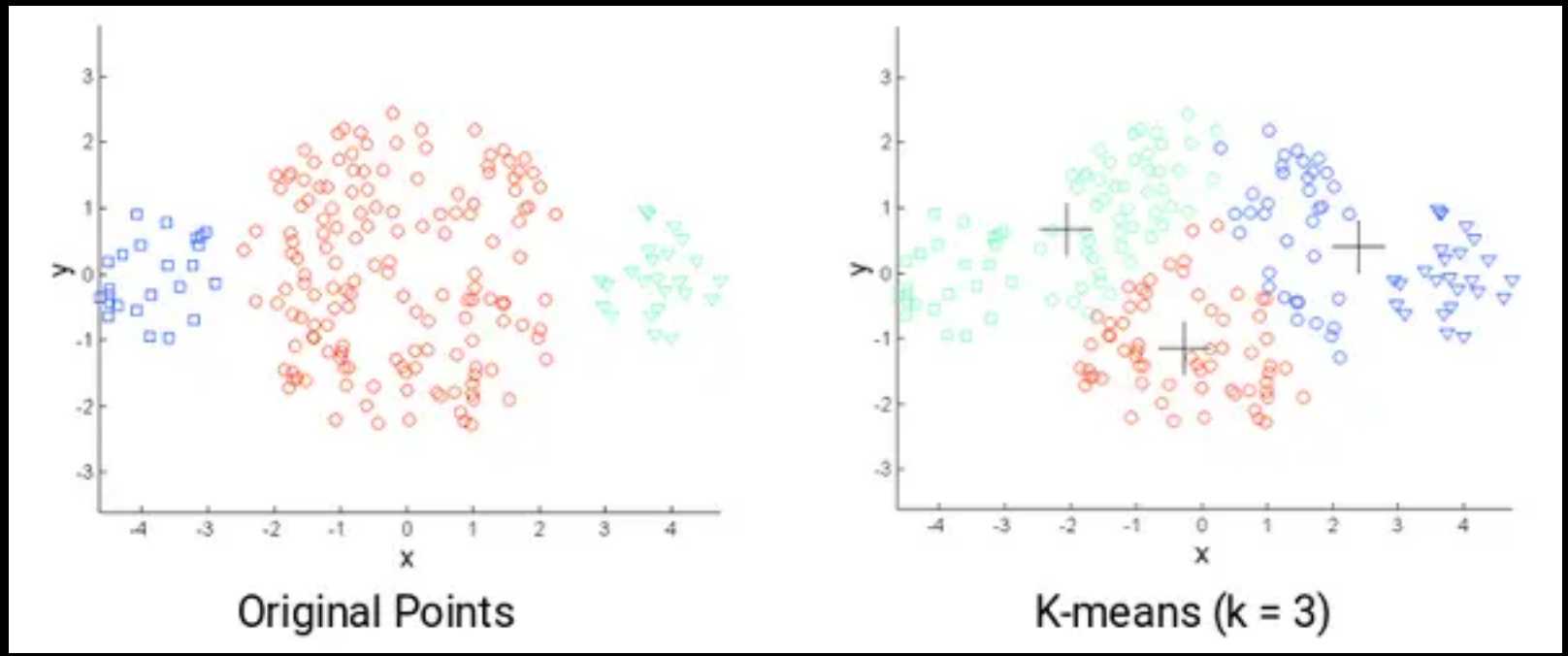
Also, one of the common challenges we face while working with K-Means is that the size of clusters is different. Let's say we have the following points:

The leftmost and the rightmost clusters are of smaller size compared to the central cluster.



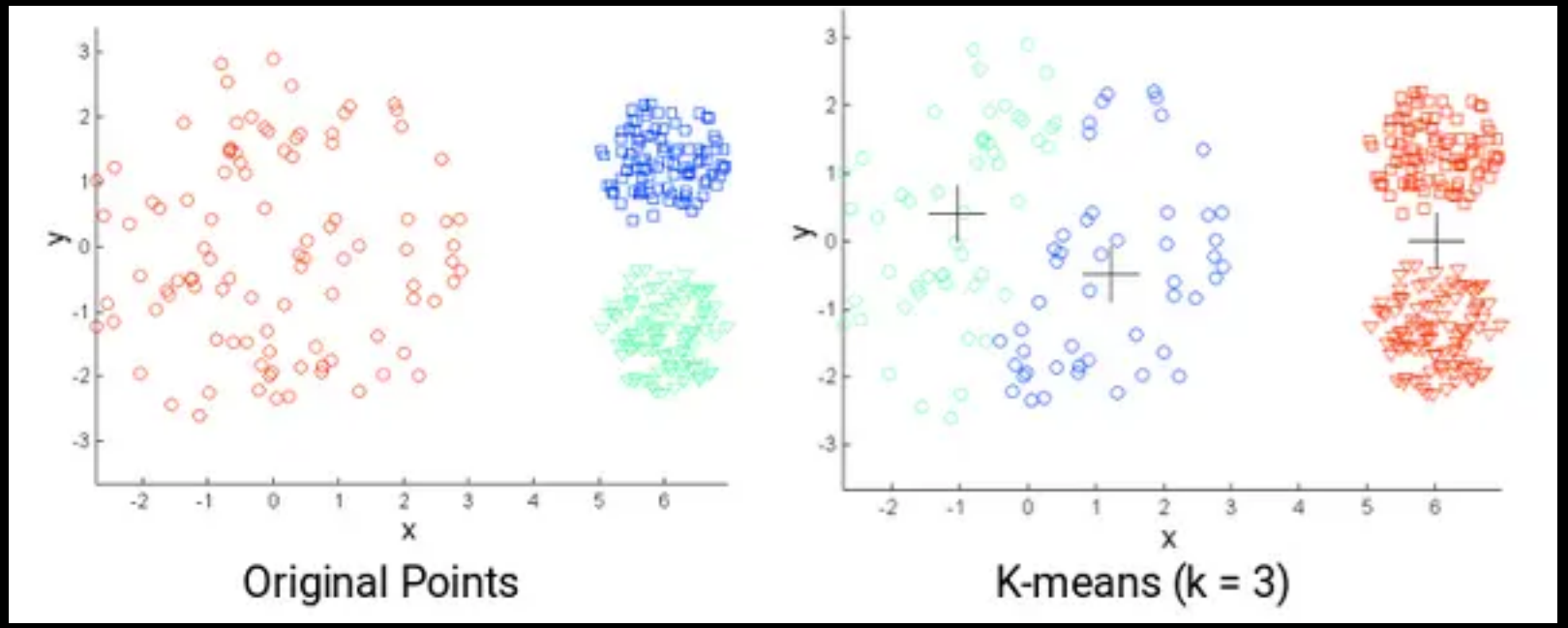
Challenges With the K-Means

Challenges



Challenges With the K-Means

Challenges



Challenges With the K-Means

Challenges

➢ We observe that tightly packed points are grouped into one cluster, while loosely spread points, previously in the same cluster, are now assigned to different clusters.

➢ Not ideal, so what can we do about this?

Challenges With the K-Means

Solutions

- Determining the optimal number of clusters for k-means clustering can be another challenge.

- Optimal number heavily relies on subjective interpretations and the underlying structure of the data. One commonly used method to find the optimal number of clusters is the elbow method, which plots the sum of squared Euclidean distances between data points and their cluster center and chooses the number of clusters where the change in the sum of squared distances begins to level off.

Challenges With the K-Means

Solutions

- Outliers can have a significant impact on the results of k-means clustering, as the algorithm is sensitive to extreme values.

- This makes it important to identify and handle outliers before applying k-means clustering to ensure that the results are meaningful and not skewed by the presence of outliers. There are various methods to identify and handle outliers, such as removing them, transforming them, or using a robust variant of k-means clustering that is less sensitive to the presence of outliers.

Challenges With the K-Means

Solutions

➢ One of the solutions is to use a higher number of clusters.

➢ So, in all the above scenarios, instead of using 3 clusters, we can have a bigger number. Perhaps setting k=10 might lead to more meaningful clusters.

➢ Remember how we randomly initialize the centroids in k-means clustering? Well, this is also potentially problematic because we might get different clusters every time. So, to solve this problem of random initialization, there is an algorithm called K-Means++ that can be used to choose the initial values, or the initial cluster centroids, for K-Means.

Content

- ~~What is K-Means Algorithm?~~
- ~~How does the K-Means Algorithm Work?~~
- ~~Challenges With the K-Means Clustering Algorithm~~
- K-means++
- Choosing right K
- Python Implementation

K-means++

Steps to Initialize the Centroids Using K-Means++

1. The first cluster is chosen uniformly at random from the data points we want to cluster. This is similar to what we do in K-Means, but instead of randomly picking all the centroids, we just pick one centroid here
2. Next, we compute the distance (D(x)) of each data point (x) from the cluster center that has already been chosen
3. Then, choose the new cluster center from the data points with the probability of x being proportional to $(D(x))^2$
4. We then repeat steps 2 and 3 until k clusters have been chosen

K-means++

Point	x	y
A	1	2
B	2	1
C	5	4
D	6	5
E	9	8

Suppose we randomly pick **A(1, 2)**.
So: C_1=(1,2)

Squared Euclidean distance is used here

Point	Distance² to C_1 (1,2)
A	0
B	$(2-1)^2 + (1-2)^2 = 2$
C	$(5-1)^2 + (4-2)^2 = 20$
D	$(6-1)^2 + (5-2)^2 = 34$
E	$(9-1)^2 + (8-2)^2 = 100$

$$P(x_i) = \frac{D(x_i)^2}{\sum_j D(x_j)^2}$$

Sum of distances² = 0 + 2 + 20 + 34 + 100 = **156**

Point	Distance²	Probability
A	0	0
B	2	2/156 = 0.013
C	20	20/156 = 0.128
D	34	34/156 = 0.218
E	100	100/156 = 0.641

K-means++

Point	x	y
A	1	2
B	2	1
C	5	4
D	6	5
E	9	8

$$P(x_i) = \frac{D(x_i)^2}{\sum_j D(x_j)^2}$$

Point	dist² to C_1	dist² to C_2	min(dist²)
A	0	100	0
B	2	85	2
C	20	25	20
D	34	18	18
E	100	0	0

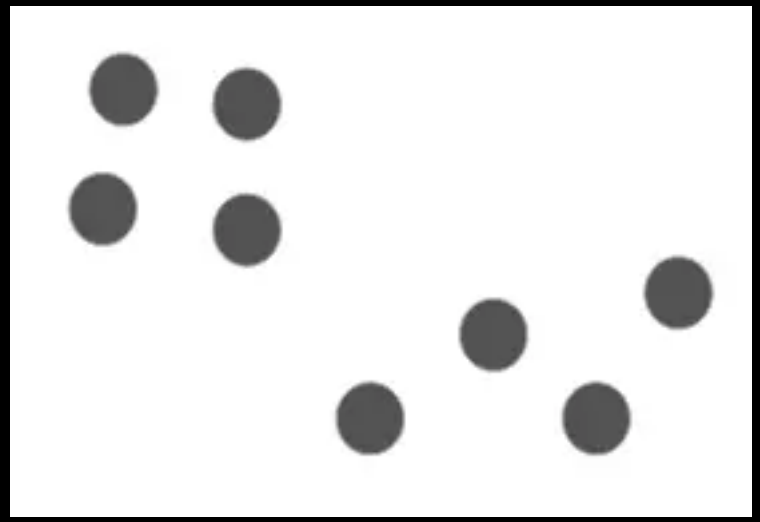
Suppose we randomly pick **A(1, 2)**.
So: C_1=(1,2)

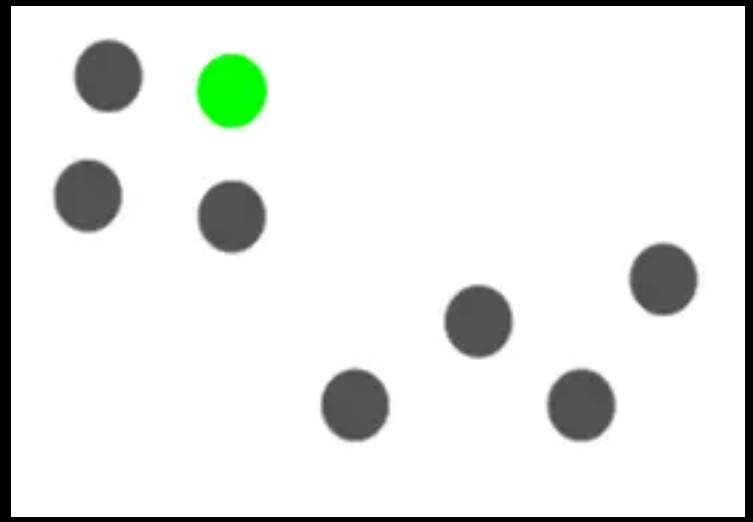
Second K: C2 = (9, 8) = E(9, 8)
Squared Euclidean distance is used here

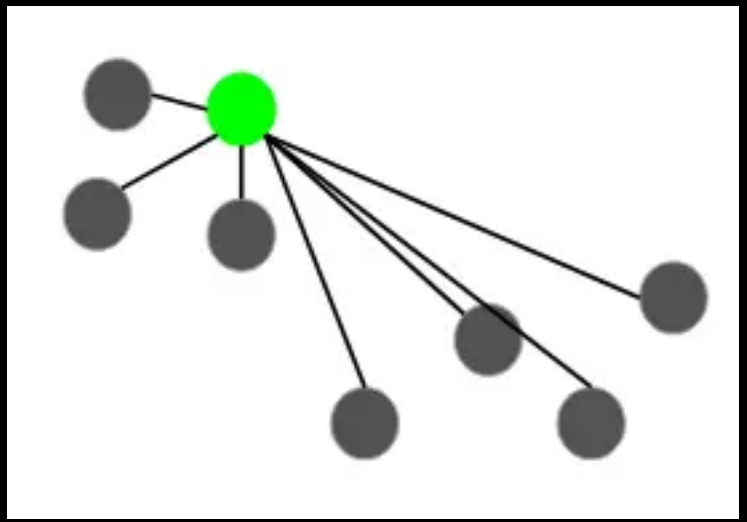
So the next centroid will likely be **C(5,4)** or **D(6,5)**

K-means++

Let’s say we have the following points, and we want to make 3 clusters here:

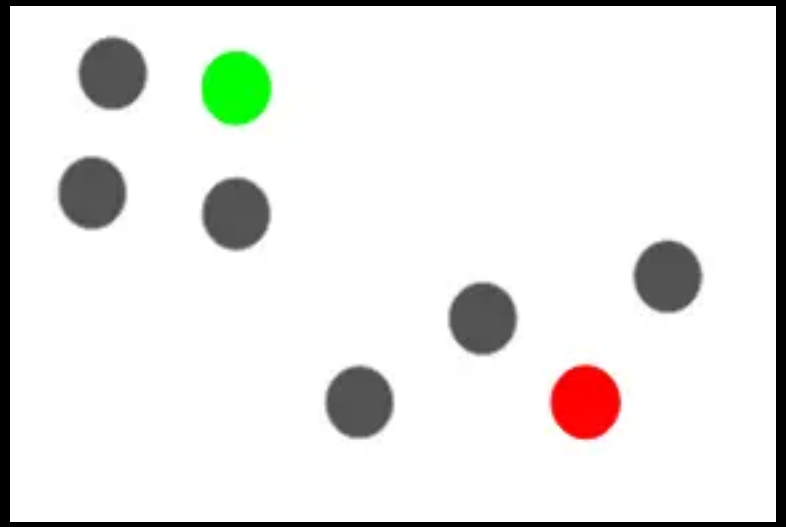


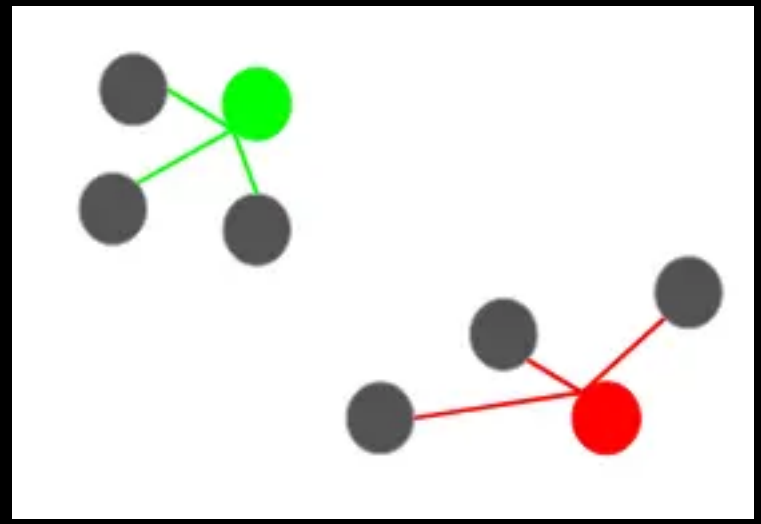


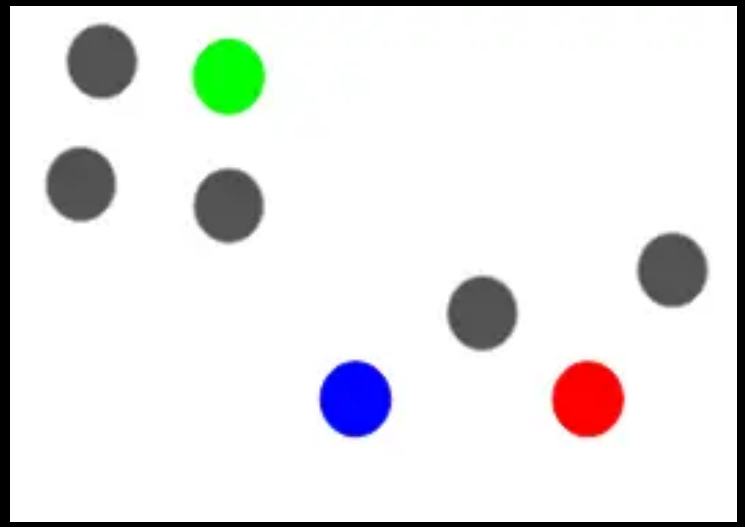


K-means++

Let’s say we have the following points, and we want to make 3 clusters here:







K-means++

- We can continue with the K-Means algorithm after initializing the centroids. Using K-Means++ to initialize the centroids tends to improve the clusters.

- Although it is computationally costly relative to random initialization, subsequent K-Means often converge more rapidly.

Content

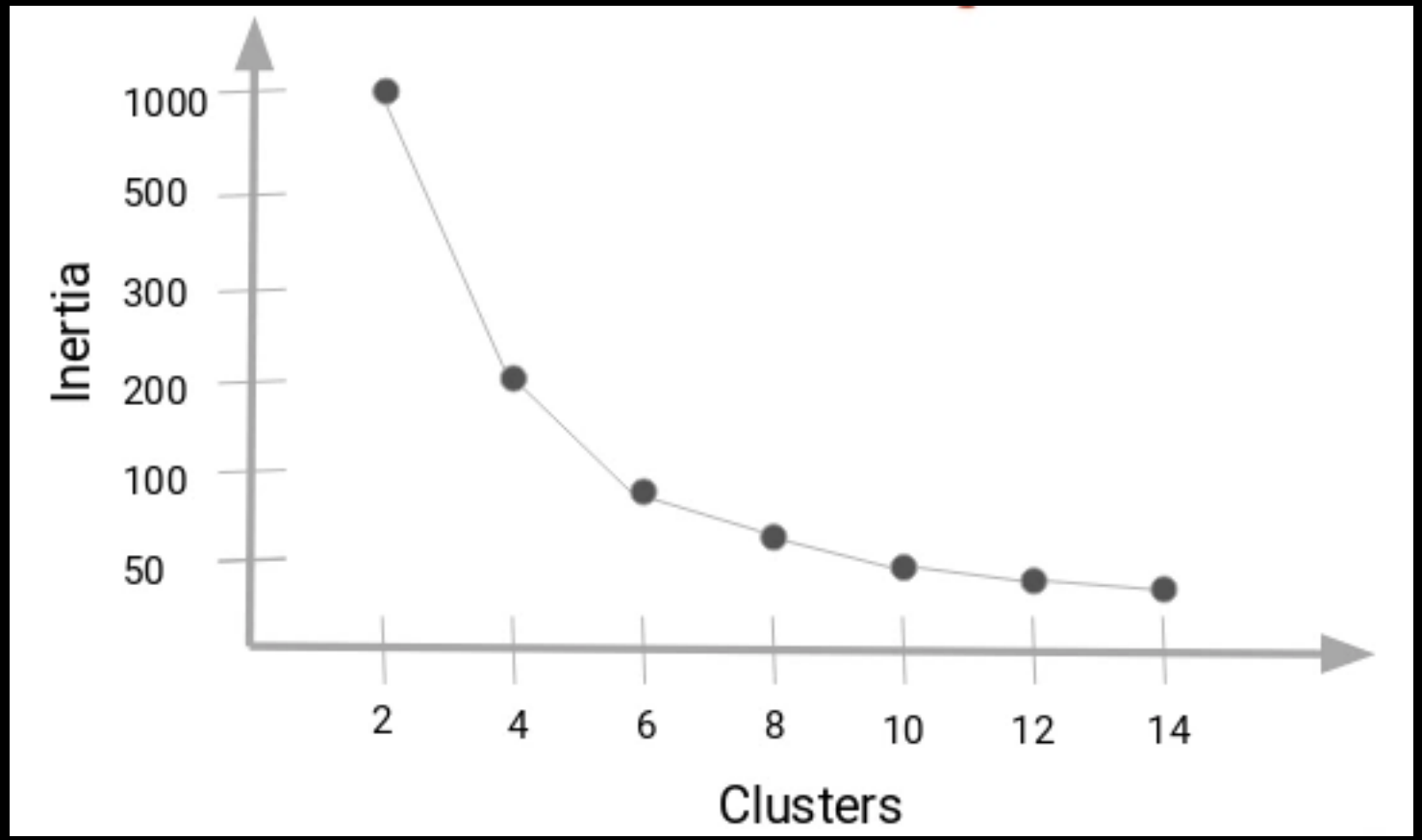
- ~~What is K-Means Algorithm?~~
- ~~How does the K-Means Algorithm Work?~~
- ~~Challenges With the K-Means Clustering Algorithm~~
- ~~K-means++~~
- Choosing right K
- Python Implementation

Choosing right K

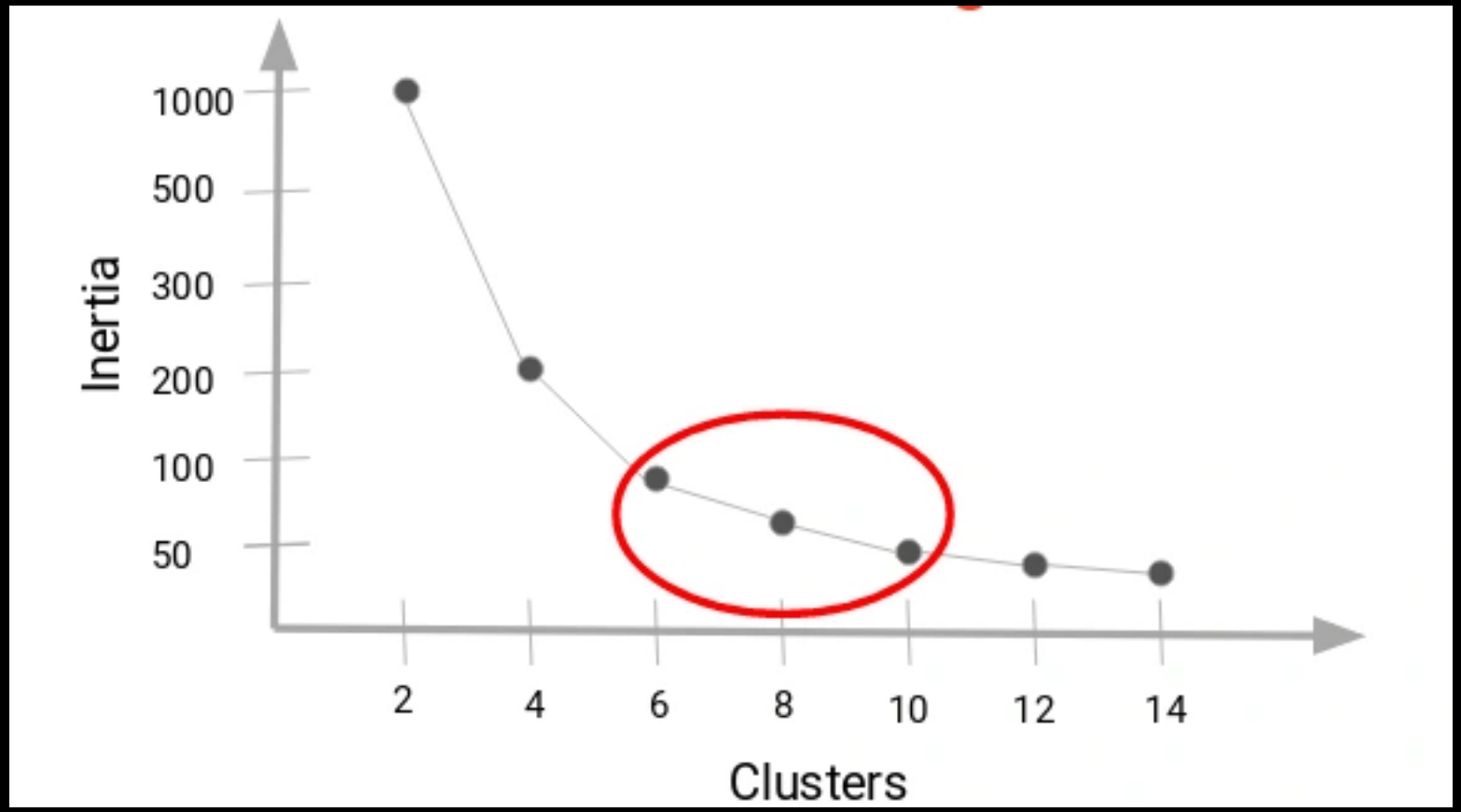
➢ But then, how can we decide the optimum number of clusters?

➢ One thing we can do is plot a graph, also known as an elbow curve, where the x-axis will represent the number of clusters and the y-axis will be an evaluation metric.

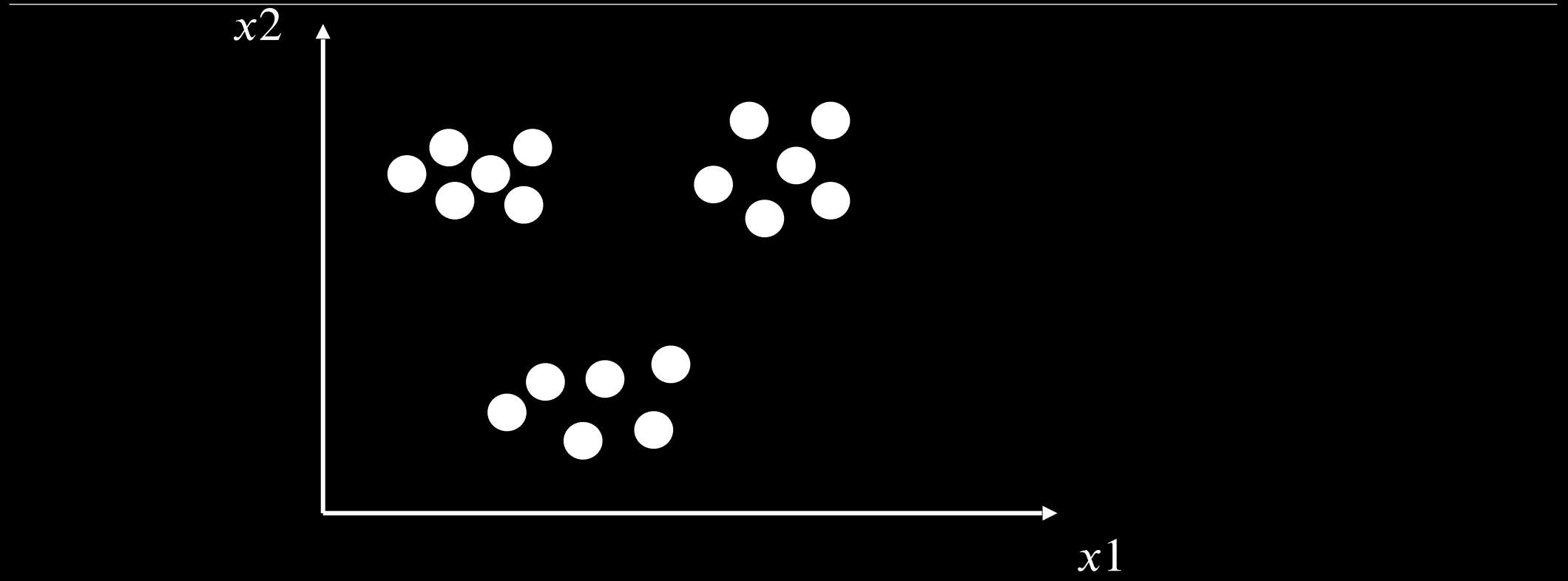
Choosing right K



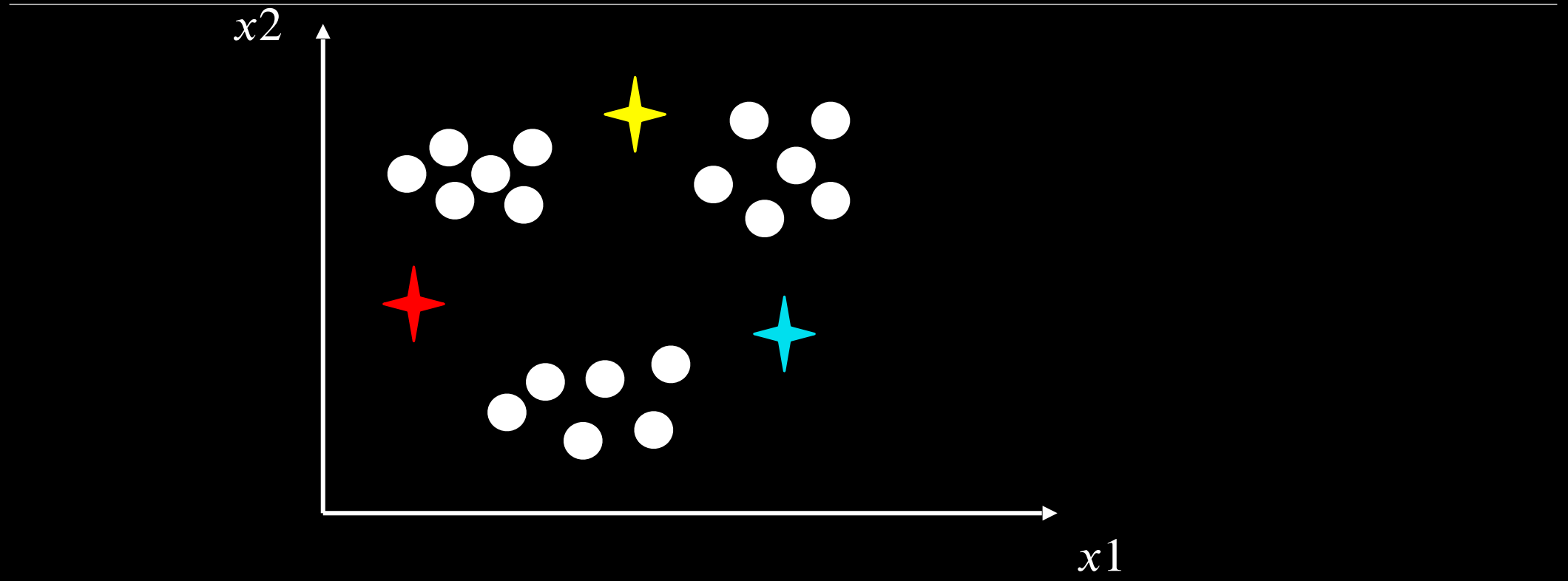
Choosing right K



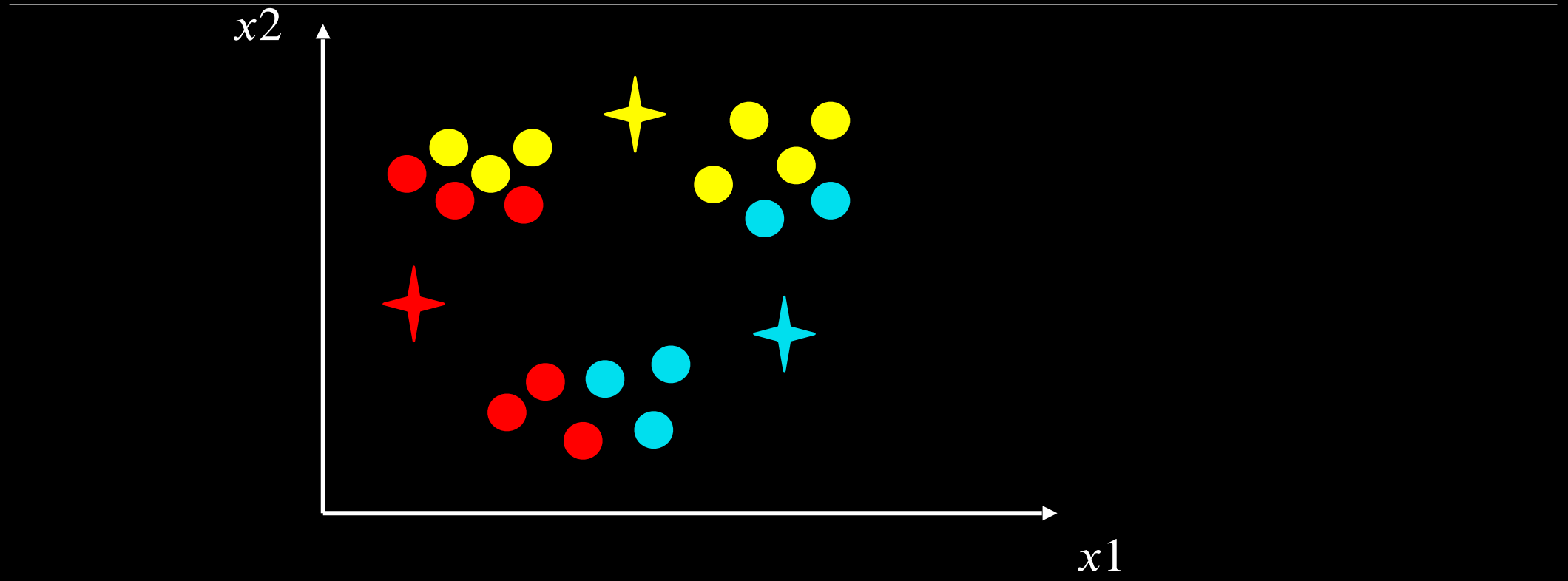
How does the K-Means Algorithm Work?



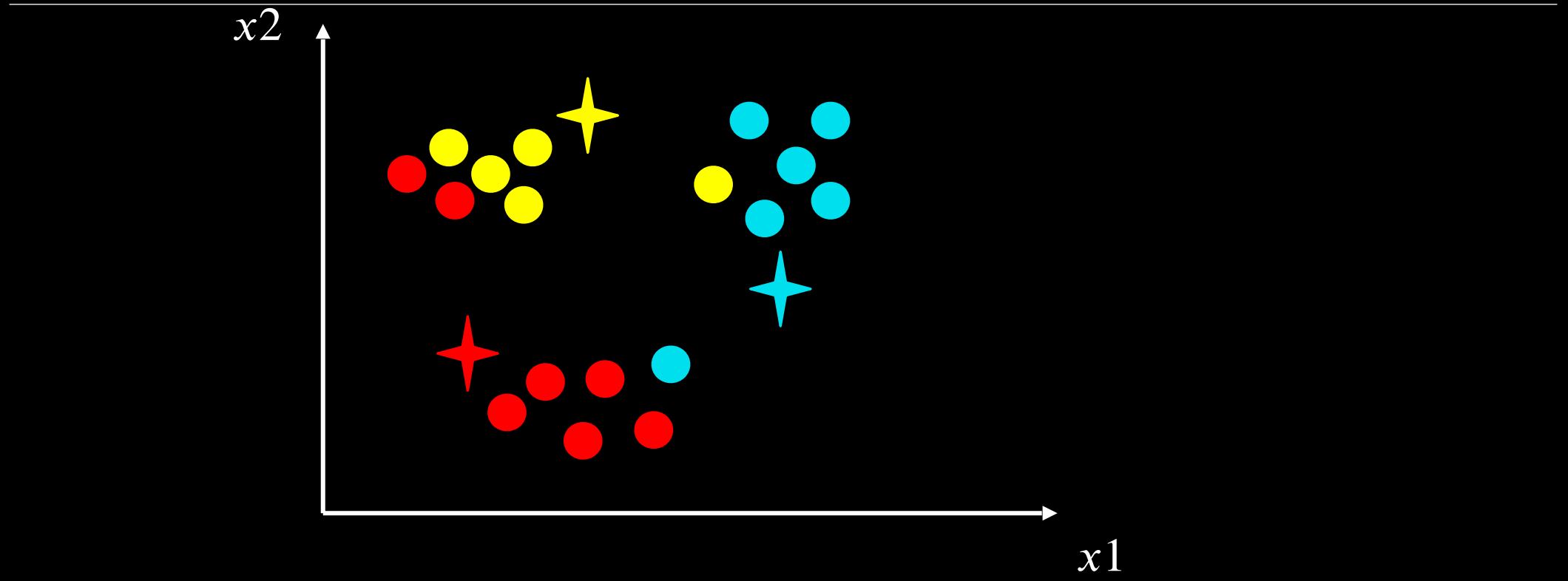
How does the K-Means Algorithm Work?



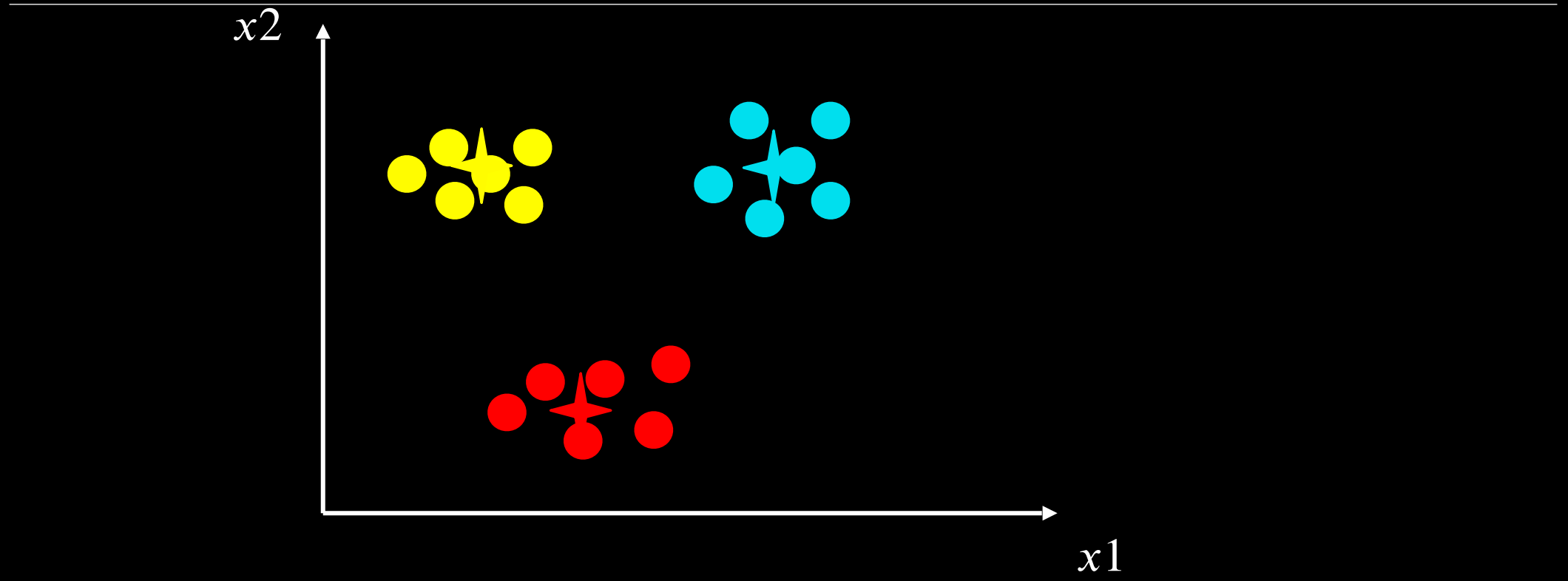
How does the K-Means Algorithm Work?



How does the K-Means Algorithm Work?



How does the K-Means Algorithm Work?



Content

- ~~What is K-Means Algorithm?~~
- ~~How does the K-Means Algorithm Work?~~
- ~~Challenges With the K-Means Clustering Algorithm~~
- ~~K-means++~~
- ~~Choosing right K~~
- Python Implementation




THANK YOU